

```
--- djangoapi\core\testing\test_djangoLogin.py ---
```

```
from django.test import TestCase, Client
from django.urls import reverse
from django.contrib.auth.models import User
```

```
from core.myLib.manageUsers import createUser
```

```
"""
```

Django tests are no done in the same way that DRF tests. They are similar but not equal for example requests and responses are different objets, and you inherit from other classes

In the apps (core in this case), all tests module names must start by test_*
In the apps (core in this case), all tests must be in the folder testing,
and there has to be the file __init__.py

To create a Django test:

You must inherit from TestCase

You must setup the class with the setUp method. Here you can create users, etc.

You must use reverse to get the url of the endpoint:

```
url = reverse('core_login')
```

The string core_login is the one you set in the urls.py of the app:

```
path('login/', views.LoginView.as_view(),name="core_login"),
```

To execute all tests in this module:

```
python manage.py test core.tests.test_djangoLogin
```

To execute all tests in all apps:

```
python manage.py test
```

Every time you run the tests a temporary database is created and the migrations are done. If you want to avoid this, you can create a test database and use it in the tests:

1. Create the test database

```
python manage.py create_db_test.py
```

create_db_test.py perform the following steps:

- Creates the test_databaseName
- If the database exists it is deleted and recreated. Every time you change a model you must recreate the test database.
- Executes the initdb.sh script. This create the migrations and migrates them.

2. Use the parameter --keepdb

```
python manage.py test --keepdb
```

The parameter --keepdb provokes that in settigns.py the test_database name do not be created, but reused. See the TESTING variable in settigns.py

Response status codes:

200 OK Petición exitosa (valor por defecto).
201 Created Se ha creado un recurso (ej. un nuevo usuario).
302 Redirected request to an other url
400 Bad Request Datos enviados inválidos o mal formateados.
401 Unauthorized El usuario no está autenticado.
403 Forbidden Autenticado, pero sin permisos para esa acción.
404 Not Found El recurso solicitado no existe.
500 Internal Server Error Error inesperado en tu código Python.

"""

```
class LoginViewTests(TestCase):
```

```
    def setUp(self):
```

```
        # Creamos un usuario real para asegurarnos de que el fallo
```

```
        # sea por la contraseña y no porque el usuario no exista.
```

```
        self.client=Client()
```

```
                                self.user =
```

```
createUser(username='test',email='test@gmail.com',password='123456789', is_active=True,  
is_superuser=False)
```

```
    def test_wrong_password(self):
```

```
        """
```

```
        Prueba que un intento de login con contraseña incorrecta
```

```
        no autentique al usuario.
```

```
        """
```

```
        #url="http://localhost:8000/core/login/"
```

```
        # 1. Intentamos loguearnos con una contraseña que NO es la suya
```

```
        # path('login/', views.LoginView.as_view(),name="core_login"),
```

```
        url = reverse('core_login')
```

```
        d={'username': 'unexisting','password': 'wrong_password'}
```

```
        response = self.client.post(url, d, format='json')
```

```
        # Obtenemos el diccionario de la respuesta.
```

```
        # Note: in DRF would have been: json_data=response.data
```

```
        json_data = response.json()#{ "ok":False,"message": "Wrong user or password",
```

```
"data":[]}
```

```
        print("-----TEST 1-----")
```

```
        print('core.LoginViewTests.test_wrong_password')
```

```
        print(f'Request url: {url}')
```

```
        print(f'Method: POST')
```

```
        print(f'Sent data: {d}')
```

```
        print(f'Response: {response}')
```

```
        print(f'Response status: {response.status_code}')
```

```
        print(f'json_data: {json_data}')
```

```
        # Verification tests
```

```
        self.assertEqual(response.status_code, 400)
```

```
        self.assertFalse(json_data['ok'])
```

```

self.assertEqual(json_data['message'], "Wrong user or password")
self.assertEqual(json_data['data'], [])

def test_correct_password(self):
    """
    Prueba que un intento de login con contraseña correcta
    autentique al usuario.
    """
    #url="http://localhost:8000/core/login/"
    # 1. Intentamos loguearnos con una contraseña que NO es la suya

    # path('login/', views.LoginView.as_view(),name="core_login"),
    url = reverse('core_login')
    d={'username': 'test','password': '123456789'}
    response = self.client.post(url, d, format='json')
    # Obtenemos el diccionario de la respuesta.
    # Note: in DRF would have been: json_data=response.data
    json_data = response.json()#{"ok":True,"message": "User test logged in",
"data":[{"username": "test"}]

    print("-----TEST 2-----")
    print('core.LoginViewTests.test_correct_password')
    print(f'Request url: {url}')
    print(f'Method: POST')
    print(f'Sent data: {d}')
    print(f'Response: {response}')
    print(f'Response status: {response.status_code}')
    print(f'json_data: {json_data}')

    # Verification tests
    self.assertEqual(response.status_code, 200)
    self.assertTrue(json_data['ok'])
    self.assertEqual(json_data['message'], "User test logged in")
    self.assertEqual(json_data['data'], [{"username": "test"}])

def test_core_logout_user_not_logged_in(self):
    # The view is protected by LoginRequiredMixin
    # In this test the user is not loggedin
    # If the user is not loggedin is redirect to /core/not_loggedin/
    #
    #Response: <HttpResponseRedirect status_code=302, "text/html; charset=utf-8",
url="/core/not_loggedin/?next=/core/logout/">

    url = reverse('core_logout')
    d={'username': 'test'}
    response = self.client.post(url, d, format='json')
    # Obtenemos el diccionario de la respuesta.
    # Note: in DRF would have been: json_data=response.data

    print("-----TEST 3-----")
    print('core.LoginViewTests.test_core_logout_user_not_logged_in')
    print(f'Request url: {url}')
    print(f'Method: POST')

```

```

print(f'Sent data: {d}')
print(f'Response: {response}')
print(f'Response status: {response.status_code}')

self.assertEqual(response.status_code, 302)

def test_core_logout_user(self):
    # The view is protected by LoginRequiredMixin
    # In this test the user is loggedin

    #This initializes session for the user
    self.client.login(username='test', password='123456789')

    url = reverse('core_logout')
    d={'username': 'test'}
    response = self.client.post(url, d, format='json')
    # Obtenemos el diccionario de la respuesta.
    # Note: in DRF would have been: json_data=response.data
    json_data = response.json()#{"ok":True,"message": "The user test is now logged
out", "data":[]}]

    print("-----TEST 4-----")
    print('core.LoginViewTests.test_core_logout_user_not_logged_in')
    print(f'Request url: {url}')
    print(f'Method: POST')
    print(f'Sent data: {d}')
    print(f'Response: {response}')
    print(f'Response status: {response.status_code}')
    print(f'json_data: {json_data}')

    # Verification tests
    self.assertEqual(response.status_code, 200)
    self.assertTrue(json_data['ok'])
    self.assertEqual(json_data,{"ok":True,"message": "The user test is now logged
out", "data":[]}]

```